Replicación de GAIL

Asier Corral e Ibon Erdoiza

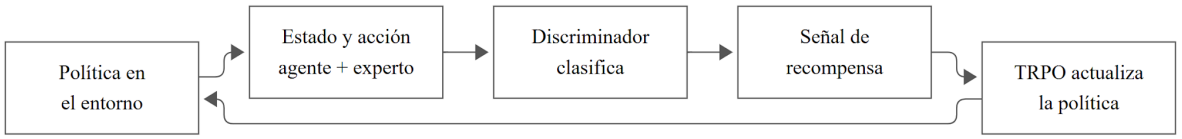
1. Introducción

Replicamos GAIL, de Ho y Ermon (2016), y lo comparamos con Behavioral Cloning (BC) usando distintas cantidades de demostraciones. Un experto entrenado con TRPO y una política aleatoria sirven de referencia. Probamos cinco entornos; CartPole es el principal y el único que cumple la regla 5/20: prueba breve en menos de 5 minutos y experimento completo en menos de 20.

Entorno	Acciones	Partidas por conjunto	Iteraciones GAIL
CartPole	Discretas	1, 4, 7, 10	20
MountainCar	Discretas	1, 4, 7, 10	300
Acrobot	Discretas	1, 4, 7, 10	300
Reacher	Continuas	4, 11, 18, 25	500
HalfCheetah	Continuas	4, 11, 18, 25	500

2. El paper en resumen

El problema es aprender a partir de demostraciones sin disponer de la recompensa de la tarea. GAIL busca reproducir el comportamiento del experto: qué estados visita y qué acciones realiza. El discriminador distingue las muestras del agente y del experto; su salida define una recompensa que TRPO utiliza para actualizar la política.

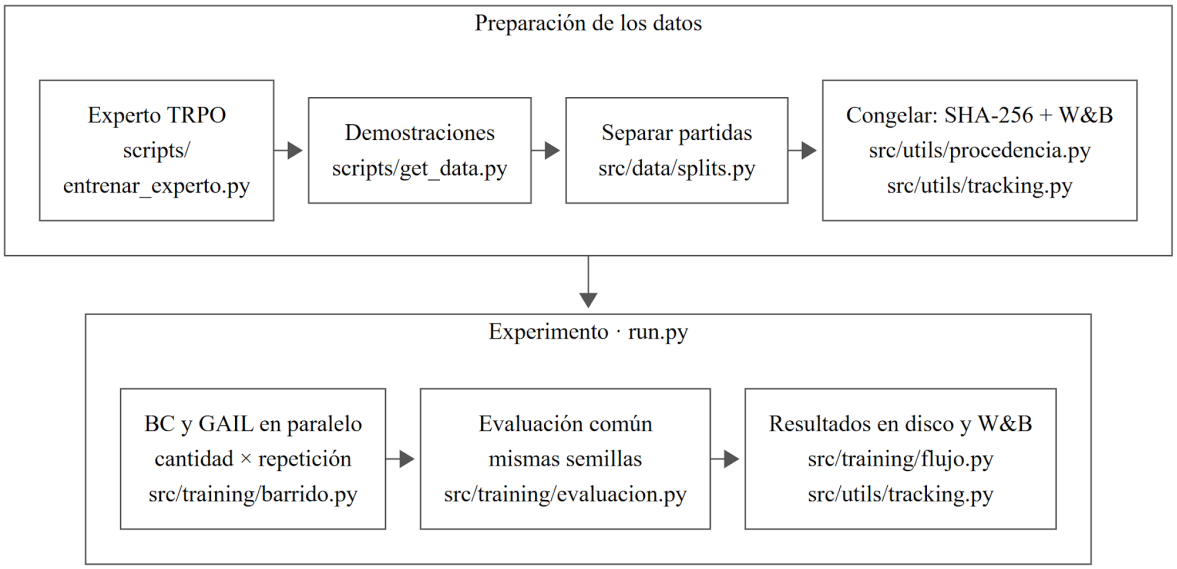


BC	GAIL
Aprendizaje supervisado sobre pares (estado, acción).	Entrenamiento alternado de la política y el discriminador.
Necesita demostraciones para entrenar.	Necesita demostraciones e interacción con el entorno.
Con pocos datos, los errores llevan al agente a estados poco representados: deriva de covariables.	Requiere entrenar dos modelos y generar nuevas partidas.

El artículo compara GAIL con BC y otros dos métodos de imitación, FEM y GTAL, en nueve tareas. Varía las demostraciones y evalúa con 50 partidas. En los Cartpole, Acrobot y Reacher, también usa 7 políticas inicializadas al azar.

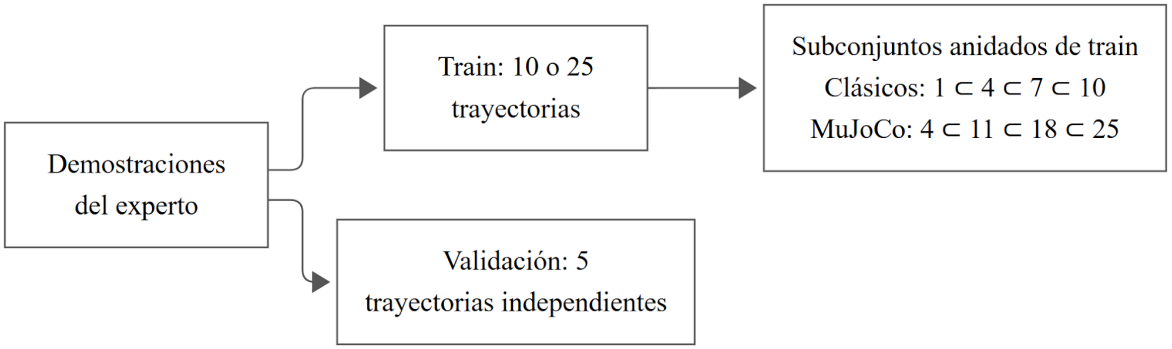
3. Arquitectura y funcionamiento

La preparación genera el experto, las demostraciones y los splits. Los experimentos reutilizan estos datos congelados, verificados mediante huellas SHA-256.

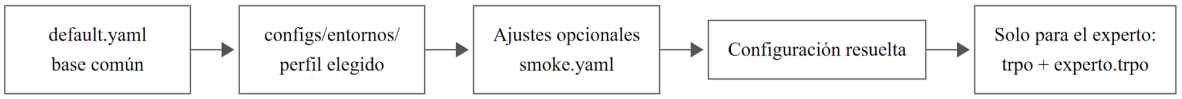


3.1. Separación de los datos

La separación entre train y validación se realiza por trayectorias completas. Los subconjuntos de train son anidados: cada uno contiene al anterior.



3.2. Configuración por capas



La configuración se combina en orden: base, perfil y ajustes opcionales. experto.trpo aplica una configuración específica al experto; smoke reduce el experimento para verificar el flujo.

3.3. Organización y reproducibilidad

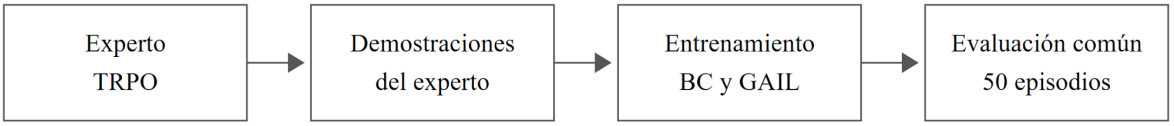
Carpeta o archivo	Responsabilidad
run.py	Coordina el experimento completo.
src/data	Entornos, demostraciones y separación de partidas.
src/models	Políticas de BC y discriminador.
src/training	TRPO, BC, GAIL, barridos y evaluación.
src/utils	Configuración, semillas, huellas y registro.
scripts	Permite ejecutar las etapas por separado.
configs	Base, perfiles de entorno y prueba smoke.
tests	Comprueba los componentes y el flujo completo.

Las semillas se inicializan solo en src/utils/seed.py y cada proceso utiliza un hilo de PyTorch. Se verifican las huellas SHA-256 de los datos congelados y se registran los artefactos en W&B. El README describe la ejecución.

4. Diferencias con el paper original

Replicamos Generative Adversarial Imitation Learning (Ho y Ermon, 2016), con adaptaciones en los entornos y en el presupuesto de entrenamiento.

Comparamos BC y GAIL con el experto TRPO y una política aleatoria, bajo un protocolo de evaluación común.



Las tablas distinguen los cambios respecto al artículo de los detalles tomados de openai/imitation y las decisiones propias.

4.1. Cambios respecto al artículo

Artículo original	**Nuestro proyecto**	**Motivo**
9 tareas; comparación con BC y otros dos métodos (FEM y GTAL).	5 tareas: CartPole, MountainCar, Acrobot, Reacher y HalfCheetah. GAIL frente a BC, con experto y agente aleatorio como referencias.	Cómputo disponible y dificultad creciente.
CartPole-v0, Acrobot-v0, MountainCar-v0, Reacher-v1 y HalfCheetah-v1.	Gymnasium 1.3 y MuJoCo 3: CartPole-v1, Acrobot-v1, MountainCar-v0, Reacher-v5 y HalfCheetah-v5.	Las versiones antiguas de Acrobot, Reacher y HalfCheetah ya no existen. CartPole: motivo pendiente.
CartPole: 300 iteraciones de GAIL.	20 iteraciones de GAIL.	Límite de tiempo de la asignatura (regla 5/20).
7 entrenamientos por configuración en los clásicos y Reacher.	5 en CartPole, MountainCar y Acrobot; 1 en Reacher y HalfCheetah.	Ejecutar cinco entrenamientos a la vez; en Reacher y HalfCheetah, seguir la pauta del código original.
Reacher: 200 iteraciones; 4, 11 o 18 demostraciones. Regularización de entropía: 0; 0,001; 0,01.	500 iteraciones; 4, 11, 18 o 25 demostraciones. Sin regularización de entropía.	Adoptamos los tamaños de otras tareas MuJoCo: openai/imitation no incluye un flujo propio para Reacher. La biblioteca no incluye esa regularización.
BC: 70 % de pares para train; 30 % para validación.	Todo el subconjunto de train; 5 trayectorias independientes para validación.	Evitar que una misma trayectoria aporte muestras a ambos conjuntos.
BC: parar cuando el error de validación deja de bajar.	20 épocas sin mejora; máximo de 500 épocas.	Concretar el criterio de parada.

4.2. Decisiones de implementación

Para los detalles que el artículo no especifica, tomamos como referencia openai/imitation o adoptamos decisiones propias, indicadas en la tabla.

Componente	Implementación	Origen
Experto	Entrenamos con TRPO. En los clásicos usamos la configuración de GAIL; en Reacher y HalfCheetah, la de RL Zoo, sin normalizar la recompensa. Conservamos el modelo con mejor evaluación.	Elección propia
Recompensa de GAIL	La recompensa se obtiene de la salida del discriminador. En MountainCar y Acrobot usamos la variante de openai/imitation que evita favorecer episodios más largos.	Código original
Normalización del discriminador	Ajustamos la normalización con los pares expertos de train y la mantenemos fija. openai/imitation la actualiza también con las muestras del agente.	Elección propia
Red de valor	Usamos la red de valor de sb3-contrib y la reajustamos tras actualizar el discriminador. Difieren el procedimiento de ajuste y el cálculo de los objetivos.	Elección propia

Criterio de parada del experto

El entrenamiento del experto se detiene al alcanzar la puntuación objetivo: 500 en CartPole, −90 en MountainCar, −70 en Acrobot, −3,5 en Reacher y 5000 en HalfCheetah.